

Halo communication in the mEVP sea-ice solver of the FESOM2 GPU port

N. Koldunov, 6 August 2026 — summary of measurements

The scheme, and the two ways of writing it

mEVP advances the ice momentum in 120 sub-cycles per model time step, and the ice-velocity halo is at present exchanged after each of them. The *wide halo* (Danilov) exchanges every K -th sub-cycle instead and recomputes a K -deep ring of ghost cells in between, so that no value is ever stale and the every-sub-cycle trajectory is reproduced exactly. The *divergence form* (Danilov) carries $\nabla \cdot \sigma$ at nodes rather than σ at elements, which halves the message the wide halo has to send.

The ring computation can be written two ways, and this turned out to matter more than either proposal: as **ring kernels** of its own, or as a **widening of the loops that already exist**. Both compute the same ring and put identical traffic on the wire.

Cost, measured against the model step and against the ice

Sea ice is 5 to 13% of a model step, so an ice-only change is divided by about ten before it reaches the step. The share of the *ice cost* — ice, ice dynamics and ice advection, computation together with time spent waiting for communication — is the measure appropriate to a sea-ice scheme, and is what the table gives.

change of the sea-ice cost at $K=8$ each mesh at its operating point	CORE2 1 node / 4 GPUs	fArc 4 / 16	DARS 8 / 32	NG5 16 / 64
reference: sea-ice cost per step	15.4 ms	26.3 ms	43.0 ms	38.7 ms
wide halo, standard form, ring kernels	−10.4%	−11.4%	−22.8%	−11.1%
wide halo, divergence form, ring kernels	+7.8%	−5.7%	−28.1%	−15.5%
wide halo, standard form, widened loops	−57.1%	−36.9%	−43.0%	−33.3%
wide halo, divergence form, widened loops	−57.8%	−43.3%	−50.2%	−42.6%
best widened-loop row, as a share of the model step	−13.9%	−9.9%	−12.5%	−6.6%

On CPU the wide halo is not profitable in either writing, at any of the three core counts tried — it at least doubles the ice cost there. The divergence form *on its own*, without a wide halo, costs 2 to 17% of the ice on GPU and saves 2 to 8.5% on CPU at large core counts.

The one number behind the table

Our ring kernels add **362 GPU kernel launches per model step**, each on a slower launch path than the owned kernel it duplicates, because their argument lists cross a threshold in Kokkos above which arguments are staged through constant memory. The gap between consecutive launches is $14.3 \mu\text{s}$ for those against $4.4 \mu\text{s}$ for the owned kernels; per step the ring kernels cost 5.9 ms of inter-kernel idle against 2.5 ms of arithmetic. Folding them into the existing loops leaves 2 of the 362 on the slow path and takes the ice-dynamics region from 16.5 to 5.5 ms per step. **Nothing moved on the wire**: each pair of rows above posts identical message and byte counts.

Which form of the scheme is better depends on how the ring is written

With ring kernels the standard form leads by 18.2 points of ice cost on CORE2 and 5.7 on fArc, the divergence form leading on the two large meshes (5.3 and 4.4). With widened loops **the divergence form leads on all four meshes**, and its lead is ordered by how much mesh each GPU holds — 0.7, 6.5, 7.2, 9.3 points against 32, 40, 99, 116 thousand surface nodes per GPU, monotone across the four. It is ahead at every K tried, though a sweep across node counts has it ahead at every point but one (fArc on two GPU nodes) rather than universally. The launch structure was not creating the difference between the two forms; it was hiding it, and it fell hardest on the divergence form because its ring gather was the most expensive of the ring kernels.

How it scales

On GPU the sea-ice cost of the standard scheme does not fall with node count — it is flat or rising on every mesh (CORE2 15.6, 15.0, 22.9, 24.4, 34.0 ms at 1 to 16 nodes; fArc 24.7 to 26.9 across the same range; NG5 48.7 to 39.1 from 2 to 16). The ocean around it scales, so the model step still improves, but adding GPUs buys nothing on the ice. The widened-loop wide halo lowers that cost and partly restores its scaling: in the divergence form the ice cost *falls* where the standard scheme's rises (fArc 16.9, 15.0, 13.5, 12.8 ms at 2 to 16 nodes; NG5 34.2, 26.0, 25.6, 22.1). **The saving therefore grows with the number of GPUs** — from the smallest to the largest node count on each mesh it goes −59 to −65% of the ice on CORE2, −39 to −53% on fArc, −30 to −51% on DARS and −30 to −44% on NG5 — so the single-node-count table above understates it. On CPU the ice scales cleanly and the wide halo is a uniform penalty at every one of the fifteen points measured.

The accuracy price, and the test that replaces bit-identity

The ring-kernel writing is exact: it reproduces the every-sub-cycle trajectory bit for bit. The widened-loop writing gives that up, because a ring node is summed in the receiving process's element order rather than the owner's; after one step of 120 sub-cycles the ice velocity differs by 8 to 15 units in the last place. In place of the bit check we hold it to a 60-day test of whether the domain decomposition becomes visible in the ice: the mean $|\Delta|$ on the one-cell sub-domain boundary divided by the mean in the interior, fArc on 16 GPUs. The widened-loop writing gives 0.836 for ice thickness, 0.796 for concentration and 1.196 for ice velocity, against 0.854, 0.831 and 1.325 for two identical runs — at or below its own control on every field. Each field needs its own control because the absolute ratio is a property of the field, not of the scheme: two identical runs already score 1.33 on ice velocity.

Recommendation

For production, the wide halo at $K=8$ on GPU, in the divergence form, written as a widening of the loops that already exist. That removes 37 to 58% of the sea-ice cost and 7 to 14% of the whole model step, and its accuracy cost is last-bit. Where bit-identity is required the ring-kernel writing remains available and is exact, at 10 to 28%. On CPU we would not use the wide halo at all.